

Brute Force Simulator



Alysson Gomes — 01710204
Jonathan Mendes — 01375492
Lindalva Ferreira — 01725681
Matheus Langone — 01753331
Natalia Azevedo — 01616881
Natthan Gonçalves — 01701854
Vitor Hugo — 01698637
Sabrina Lemos — 01700691

Sobre o Projeto



Ataques Cibernéticos

Os ataques cibernéticos representam uma das maiores ameaças da era digital, afetando pessoas, empresas e governos em todo o mundo...



Método Brute Force

O método Brute Force é uma técnica utilizada para tentar descobrir senhas, chaves de criptografia ou credenciais de acesso por meio da realização de inúmeras combinações possíveis até encontrar a correta...



Conceitos Aplicados

Segurança da Informação

Autenticação de usuários

Ataques de força bruta

Estrutura cliente-servidor

Manipulação de DOM

Requisições HTTP

Integração com banco de dados

 .github/workflows	2 weeks ago
 desafio-logging	last week
 node_modules	2 weeks ago
 public	2 weeks ago
 syslog-ng/config	last week
 .gitignore	last week
 Dockerfile	2 weeks ago
 README.md	last week
 bruteforce.js	2 weeks ago
 daemon.json.backup	last week
 db.js	2 weeks ago
 docker-compose.yml	last week
 login_db.sql	2 weeks ago
 package-lock.json	2 weeks ago
 package.json	2 weeks ago



Arquitetura

Simulação de Ataque e Segurança

Simulação de Ataque

O sistema executa tentativas automáticas utilizando uma wordlist de senhas comuns.

Segurança

- Sistema de bloqueio por tentativas
- CAPTCHA



Desafio 1 — Automação de Deploy e Integração Contínua (CI/CD)



O objetivo foi criar uma esteira de Automação de Deploy e Integração Contínua (CI/CD) utilizando GitHub Actions. A arquitetura foi projetada para garantir que qualquer alteração no código seja testada, empacotada em um container estável e disponibilizada para produção de forma 100% automática, rápida e segura.



Resultados



name: CD

on:

push:

branches: [main]

jobs:

build-and-push:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Login Docker Hub

uses: docker/login-action@v3

with:

username: \${ secrets.DOCKERHUB_USERNAME }

password: \${ secrets.DOCKERHUB_TOKEN }

- name: Build and push image

uses: docker/build-push-action@v5

with:

push: true

tags: \${ secrets.DOCKERHUB_USERNAME }/projeto-devops:lates

t

Pipeline CI/CD automatizado

Validação automática de alterações

Empacotamento padronizado da aplicação

Deploy automatizado

Redução de falhas manuais



Resultados



name: CI

on:

push:

branches: ["**"]

pull_request:

branches: [main, develop]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Setup Node.js

- uses: actions/setup-node@v4

- with:

- node-version: '22.20.0'

- cache: 'npm'

- name: Install dependencies

- run: npm install

- name: Run lint

- run: npm run lint || true

- name: Run tests

- run: npm test -- --passWithNoTests

Pipeline CI/CD automatizado

Validação automática de alterações

Empacotamento padronizado da aplicação

Deploy automatizado

Redução de falhas manuais



Desafio 2 — Docker, Monitoramento e Dashboard DevOps



Durante o desenvolvimento do projeto DevOps, fui responsável pela implementação e organização do ambiente da aplicação utilizando tecnologias modernas de infraestrutura e monitoramento.



Principais atividades

Integração completa do Docker ao projeto

Configuração do Dockerfile

Configuração do docker-compose.yml

Containerização da aplicação e banco MySQL

Ajustes de conectividade entre serviços

Funcionalidades

Monitoramento de CPU

Monitoramento de Memória RAM

Monitoramento de Disco

Status do Sistema

Gráficos dinâmicos

Atualização automática

Outros...



Resultados



FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["sh", "-c", "sleep 15 && node server.js"]

Ambiente totalmente containerizado

Dashboard funcional em tempo real

Integração com múltiplas fontes de dados

Estrutura Git organizada



Resultados



```
services:
  app:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      mysql:
        condition: service_healthy
    environment:
      DB_HOST: mysql
      DB_USER: root
      DB_PASSWORD: root
      DB_NAME: login_db

mysql:
  image: mysql:8.0
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: login_db
  ports:
    - "3306:3306"
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
    interval: 5s
    timeout: 5s
    retries: 10
```

Ambiente totalmente containerizado

Dashboard funcional em tempo real

Integração com múltiplas fontes de dados

Estrutura Git organizada



Resultados

```
● ● ●

/* Container */
.success-message {
  text-align: center;
  max-width: 500px;
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}

.success-message__icon {
  max-width: 75px;
}

/* Title */
.success-message__title {
  color: var(--success-primary);
  transform: translateY(25px);
  opacity: 0;
  transition: all 200ms ease;
}

body.active .success-message__title {
  transform: translateY(0);
  opacity: 1;
}
```

Ambiente totalmente containerizado

Dashboard funcional em tempo real

Integração com múltiplas fontes de dados

Estrutura Git organizada



Desafio 3 — Kubernetes e Helm para Deploy Multiambiente



Implementação de uma solução de orquestração de containers utilizando Kubernetes e Helm com foco em deploy multiambiente.



Resultados



```
apiVersion: v2
name: meu-app
description: A Helm chart for Kubernetes

# A chart can be either an 'application' or a 'library' chart.
#
# Application charts are a collection of templates that can be packaged into versioned archives
# to be deployed.
#
# Library charts provide useful utilities or functions for the chart developer. They're included as
# a dependency of application charts to inject those utilities and functions into the rendering
# pipeline. Library charts do not define any templates and therefore cannot be deployed.
type: application

# This is the chart version. This version number should be incremented each time you make changes
# to the chart and its templates, including the app version.
# Versions are expected to follow Semantic Versioning (https://semver.org/)
version: 0.1.0

# This is the version number of the application being deployed. This version number should be
# incremented each time you make changes to the application. Versions are not expected to
# follow Semantic Versioning. They should reflect the version the application is using.
# It is recommended to use it with quotes.
appVersion: "1.16.0"
```

Item Status Helm instalado e configurado

Cluster Kubernetes funcionando

Chart criado com templates

Deploy DEV com 1 réplica

Deploy PROD com 3 réplicas

Port-forward funcionando

Aplicação acessível via navegador

Rollback executado com sucesso



Resultados



```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "meu-chart.fullname" . }}
  labels:
    {{- include "meu-chart.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: http
      protocol: TCP
      name: http
  selector:
    {{- include "meu-chart.selectorLabels" . | nindent 4 }}
```

Item Status Helm instalado e configurado

Cluster Kubernetes funcionando

Chart criado com templates

Deploy DEV com 1 réplica

Deploy PROD com 3 réplicas

Port-forward funcionando

Aplicação acessível via navegador

Rollback executado com sucesso



Resultados



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "meu-chart.fullname" . }}
  labels:
    {{- include "meu-chart.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "meu-chart.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      {{- with .Values.podAnnotations }}
      annotations:
        {{- toYaml . | nindent 8 }}
      {{- end }}
    labels:
      {{- include "meu-chart.labels" . | nindent 8 }}
      {{- with .Values.podLabels }}
      {{- toYaml . | nindent 8 }}
      {{- end }}
```

Item Status Helm instalado e configurado

Cluster Kubernetes funcionando

Chart criado com templates

Deploy DEV com 1 réplica

Deploy PROD com 3 réplicas

Port-forward funcionando

Aplicação acessível via navegador

Rollback executado com sucesso



Desafio 4 — Syslog-ng.conf

```
@version: 3.28

options {
    chain_hostnames(off);
    flush_lines(0);
    use_dns(no);
    use_fqdn(no);
    keep_hostname(yes);
    perm(0644);
    stats_freq(3600);
};

source s_tcp {
    tcp(ip(0.0.0.0) port(514) flags(no-parse));
};

source s_udp {
    udp(ip(0.0.0.0) port(514) flags(no-parse));
};

destination d_docker {
    file("/var/log/docker/${YEAR}/${MONTH}/${DAY}/${HOST}.log");
};

log {
    source(s_tcp);
    source(s_udp);
    destination(d_docker);
    flags(flow-control);
};
```

Desenvolve um sistema como uma solução de registro centralizado utilizando Docker e Syslog-ng. Seu objetivo é coletar e armazenar os registros gerados por vários contêineres em um único local, organização cronológica: por data e origem para facilitar buscar futuras no monitoramento, auditoria e detecção de falhas no sistema.

"logs em ilhas isoladas"

Teste de Software

**Falhas
Identificadas e
Correções
Implementadas**

 **BRIGADO!**